Introduction to MATLAB

Outline

- Simple arithmetic operations
- Plotting in Matlab
- Other useful commands

Simple Arithmetic Operations

The four operators +, −, *, and/are used as usual to add, subtract, multiply, and divide two numbers. In MATLAB a left division operator\is also available, and the power of a number is computed by the operator ^.

Commands	Results	Comments
`3+5`	`ans = 8`	% Addition
`5-2`	`ans = 3`	% Subtraction
`6*7`	`ans = 42`	% Multiplication
`2/4`	`ans = 0.5`	% Division
`2\4`	`ans = 2`	% Left division
`2^3`	`ans = 8`	% Power

Simple Arithmetic Operations

Priority of the operation

Commands	Results	Comments
`3+7*2`	`ans = 17`	The multiplication is executed before the addition.
`2^2+2*(3-2*(1+14/7))`	`ans = -2`	The priority is specified from the parentheses.

Commands	Results	Comments
`0.4+1.2`	`ans = 1.6000`	Operations between decimal numbers.
`.2+.4`	`ans = 0.6000`	If the integer part of a decimal number is zero it is possible to omit it.
`-3-1`	`ans = -4`	Operations between negative numbers.

Commands	Results	Comments
`1e3`	`ans = 1000`	$1*10^3$.
`3e1`	`ans = 30`	$3*10^1$.
`4e-2`	`ans = 0.0400`	$4*10^{-2}$.
`36e9`	`ans = 3.6000e+10`	$36*10^9$. The result is written as $3.6*10^{10}$.

Simple Arithmetic Operations

Constants

Commands	Results	Comments
`pi`	`ans = 3.1416`	Mathematical constant π.
`i`	`ans = 0 + 1.0000i`	Imaginary unit i (used to define complex numbers).
`j`	`ans = 0 + 1.0000i`	Alternative representation of the imaginary unit.
`inf`	`ans = Inf`	Infinity (∞).
`-inf`	`ans = -Inf`	Minus infinity ($-\infty$).
`eps`	`ans = 2.2204e - 016`	Spacing of floating point numbers. This is the highest accuracy in which the result of an operation can be calculated.
`NaN`	`ans = NaN`	Unidentified number (not a number).
`realmax`	`ans = 1.7977e+308`	Largest positive floating point number defined in MATLAB.
`realmin`	`ans = 2.2251e - 308`	Smallest positive floating point number defined in MATLAB.

Simple Arithmetic Operations

Built-in functions

Commands	Results	Comments
`sqrt(2)`	`ans = 1.4142`	Square root.
`abs(-4)`	`ans = 4`	Absolute value.
`sign(3)`	`ans = 1`	The function *sign* returns 1 if the input number is positive, 0 if the input is zero, and −1 if the input is negative.
`exp(1)`	`ans = 2.7183`	Exponential.
`log(1)`	`ans = 0`	Natural logarithm.
`log10(100)`	`ans = 2`	Base 10 logarithm.
`log2(4)`	`ans = 2`	Base 2 logarithm.
`sin(pi/2)`	`ans = 1`	Sine.
`cos(pi)`	`ans = -1`	Cosine.
`tan(pi/4)`	`ans = 1`	Tangent.
`asin(0.5)`	`ans = 0.5236`	Arcsine (inverse sine).
`acos(-1)`	`ans = 3.1416`	Arccosine (inverse cosine).
`atan(1)`	`ans = 0.7854`	Arctangent (inverse tangent).
`sinh(0.1)`	`ans = 0.1002`	Hyperbolic sine.
`cosh(0.2)`	`ans = 1.0201`	Hyperbolic cosine.
`tanh(pi/6)`	`ans = 0.4805`	Hyperbolic tangent.
`ceil(0.6)`	`ans = 1`	Rounding toward the closer larger integer.
`floor(0.6)`	`ans = 0`	Rounding toward the closer smaller integer.
`round(0.6)`	`ans = 1`	Rounding toward the closer integer.
`fix(0.6)`	`ans = 0`	Rounding toward the closer-to-zero integer.
`factorial(5)`	`ans = 120`	Factorial function.

Simple Arithmetic Operations

Variables

Commands	Results	Comments
`theta=45*pi/180`	`theta = 0.7854`	First, we convert the 45° to radians. The result is assigned to a variable named *theta*.
`a=sin(theta)`	`a = 0.7071`	Definition of the variable *a*.
`b=cos(theta)`	`b = 0.7071`	Definition of the variable *b*.
`c=a^2+b^2`	`c = 1`	The result of the first expression is assigned to the variable *c*.
`a*b`	`ans = 0.5000`	The second expression is computed and the result is stored in the default variable *ans*.
`d=c+ans`	`d = 1.5000`	The variable *ans* can be used for further computations.
`A=6`	`A = 6`	Definition of the variable *A*.
`a=3`	`a = 3`	Definition of the variable *a*.
`c=A+a`	`c = 9`	It is obvious that there is distinction between small and capital letters.

Commands	Results	Comments
`x='hello'`	`x = hello`	Text assignment to a variable.

Simple Arithmetic Operations

Clear and clc command

The command `clear` is employed in order to delete a variable from the memory. The syntax is `clear` and the variable(s) name(s).

The command `clc` (named after the initial letters of clear command) clears the command window from anything written before. It must not be confused with the command `clear` as it does not delete anything from the memory.

Commands	Results	Comments
`a=5`	`a = 5.00`	Assign a value to variable *a*.
`clear a`		Deleting variable *a* from memory.
`a`	`??? Undefined function or variable 'a'.`	Indeed variable *a* is not defined.
`clear`		Deleting all variables from the memory.
`who`		Indeed the memory is null.

Simple Arithmetic Operations

Row vectors

To define a row vector, the elements of the vector are given into square brackets [...]. Spaces or commas are inserted between the elements. A vector element is specified by its index. The index numbering starts from 1, namely, the index of the first element of a vector is 1. In order to refer to a position in a vector, parentheses are used.

Commands	Results	Comments
`a = [1 2 3 4 5]`	`a = 1 2 3 4 5`	Row vector *a* of 5 elements.
`b = [-6,-7,-8,-9,-10]`	`b = -6 -7 -8 -9 -10`	Row vector *b* of 5 negative elements.
`c = [0.3, 8.5, 6, -2.4]`	`c = 0.30 8.50 6.00 -2.40`	Row vector *c* of 4 decimal elements.
`b(3)`	`ans = -8`	The third element of vector *b*.
`b(6)`	`??? Index exceeds matrix dimensions.`	Vector *b* has 5 elements. Thus, referring to its 6th position causes an error.

Simple Arithmetic Operations

Length / size command

In order to compute the number of elements of a vector, the appropriate command is the command `length`. A similar command is the command `size`. Command `size` is usually employed to compute the dimensions of a matrix. However, vectors are special case of matrices; thus, using the command `size` to compute the length of a vector is completely valid.

Commands	Results	Comments
`length(a)`	`ans = 5`	Number of elements of vector *a*.
`size(a)`	`ans = 1 5`	Vector *a* consists of 1 row and 5 columns.

Simple Arithmetic Operations

Replacing vector position

It is also possible to edit only one specific element of a vector by referring to the position (index) of the vector. If between the position and the rest vector elements there are no defined elements, the null positions are filled with zeros.

Commands	Results	Comments
`a`	`a = 1 2 3 4 5`	Vector *a* is already defined.
`a(3) = 99`	`a = 1 2 99 4 5`	The value 99 is inserted at the third position of vector *a*, replacing the previous value.
`a(10) = 23`	`a = 1 2 99 4 5 0 0 0 0 23`	The value 23 is inserted at the tenth position of *a*. Notice that the intermediate (6th, 7th, 8th, and 9th) positions are filled with zeros.

Simple Arithmetic Operations

Defining step of the vector

Commands	Results	Comments
`a=1:2:11`	`a = 1 3 5 7 9 11`	The elements of vector *a* start from 1, end at 11, and are spaced by 2.
`b=1:6`	`b = 1 2 3 4 5 6`	If no step is specified, by default the step considered is 1.
`c=0:0.2:1`	`c = 0 0.2 0.4 0.6 0.8 1.0`	Decimal step.
`d=3:-1:-3`	`d = 3 2 1 0 -1 -2 -3`	Negative step.
`e=0:pi/6:pi`	`e = 0 0.5236 1.0472 1.5708 2.0944 2.6180 3.1416`	Vector *e* is defined from 0 to π with step π/6.

Simple Arithmetic Operations

Operation in vector

Commands	Results	Comments
`a=1:2:11`	`a = 1 3 5 7 9 11`	The elements of vector *a* start from 1, end at 11, and are spaced by 2.
`b=1:6`	`b = 1 2 3 4 5 6`	If no step is specified, by default the step considered is 1.
`add=a+b`	`add = 2 5 8 11 14 17`	Vector *add* is the sum of vectors *a* and *b*.
`sub=a-b`	`sub = 0 1 2 3 4 5`	Subtraction of vectors.
`a.*b`	`ans = 1 6 15 28 45 66`	Multiplication element per element.
`a./b`	`ans = 1.00 1.50 1.66 1.75 1.80 1.83`	Division element per element.
`a.\b`	`ans = 1.0000 0.6667 0.6000 0.5714 0.5556 0.5455`	Left division element per element.
`a.^2`	`ans = 1 9 25 49 81 121`	Each element of *a* is raised to the power of 2.
`2.^a`	`ans = 2 8 32 128 512 2048`	2 is raised to each element of *a*.
`a.^b`	`ans = 1 9 125 2401 59049 1771561`	Each element of *a* is raised to the power of the corresponding element of *b*.

Simple Arithmetic Operations

Column vector

Commands	Results	Comments

`a=[1;2;3;4]`	`a =`
	`1`
	`2`
	`3`
	`4`
		First way of creating a column vector. The question mark is used to change row.

`b=[5`
`6`
`7`
`8]`
	`b =`
	`5`
	`6`
	`7`
	`8`
		Second way of creating a column vector. By pressing Enter when the square brackets are open also changes row.

`c=[9 10 11 12]`
`d=c'`
	`c = 9 10 11 12`
	`d =`
	`9`
	`10`
	`11`
	`12`
		Third way of creating a column vector. The transpose operator is used to convert a row vector into a column vector.

`e=d'`	`e = 9 10 11 12`	The transpose operator also converts a column vector into a row vector.
`z=[1:0.1:100]';`		Column vector `z` from 1 to 100 with step 0.1.
`size(z)`	`ans = 991 1`	Indeed `z` is a an array (or matrix) of 991 rows and 1 column.
`length(z)`	`ans = 991`	The `length` command in a column vector returns the number of elements.

Simple Arithmetic Operations

Dot product

Commands	Results	Comments
`a=1:4` `b=2:5`	`a = 1 2 3 4` `b = 2 3 4 5`	Definition of the row vectors *a* and *b*.
`dot1=dot(a,b)`	`dot1= 40`	Dot product computed by the command `dot`.
`dot2=sum(a.*b)`	`dot2= 40`	The dot product is computed according to its definition. Notice that the use of the dot operator before the multiplication is necessary.

Simple Arithmetic Operations

Useful commands

Commands	Results	Comments
`a = [4 2 7 0 6]`	`a = 4 2 7 0 6`	Definition of vector *a*.
`s = sum(a)`	`s = 19`	Sum of the elements of *a*.
`c = cumsum(a)`	`c = 4 6 13 13 19`	Cumulative sum. The result is obtained as [4, 4+2, 4+2+7, 4+2+7+0, 4+2+7+0+6].
`p = prod(a)`	`p = 0`	Product of all elements.
`d = diff(a)`	`d = −2 5 −7 6`	Difference between two consecutive elements, i.e., $d(1) = a(2) - a(1)$, etc.
`[m,i] = max(a)`	`m = 7` `i = 3`	The largest value is assigned to variable *m*, and its index is assigned to variable *i*.
`[m,i] = min(a)`	`m = 0` `i = 4`	The smaller value is assigned to variable *m*, and its index is assigned to variable *i*.
`max(a)`	`ans = 7`	If no output variable is specified, only the largest value is returned.
`mean(a)`	`ans = 3.8000`	Mean value of the elements.
`median(a)`	`ans = 4`	Median value of the vector.
`sort(a)`	`ans = 0 2 4 6 7`	Sorting in ascending order.
`sort(a,'descend')`	`ans = 7 6 4 2 0`	Sorting in descending order.

Simple Arithmetic Operations

Matrices

Commands	Results	Comments

`A= [1 2 3; 4 5 6; 7 8, 9]`	`A = 1 2 3`
	`4 5 6`
	`7 8 9`
		The matrix **A** is created. It consists of three rows and three columns.

`A= [1 2 3`
`4 5 6`
`7, 8, 9]`
	`A = 1 2 3`
	`4 5 6`
	`7 8 9`
		Second way of creating a matrix.

`D= [0:.1:.5; 0:−.1:−.5; 0:pi/5:pi]`	`D = 0 0.1 0.2 0.3 0.4 0.5`
	`0 −0.1 −0.2 −0.3 −0.4 −0.50`
	`0 0.63 1.26 1.88 2.51 3.14`
		Third way of creating a matrix.

Commands	Results		Comments
`A(1,1)`	`ans = 1`		The first element of matrix **A** (first row-first column).
`A(3,2)`	`ans = 8`		The element in the third row-second column of **A**.
`size(A)`	`ans = 3`	`3`	The dimensions of matrix **A** (3×3).

Simple Arithmetic Operations

Working with matrix

Commands	Results	Comments

`A= [1 2 3; 4 5 6; 7 8 9]`	`A = 1 2 3`
	`4 5 6`
	`7 8 9`
		Matrix **A** of size 3 × 3.

`A(:,:)`	`ans = 1 2 3`
	`4 5 6`
	`7 8 9`
		The whole matrix **A**.
		Explanation: The operator ":" means that all rows and all columns are selected.

`A(2,:)`	`ans = 4 5 6`	The second row of **A**. *Explanation*: From the rows only the second one is picked. From the columns all of them are selected.

`A(:,3)`	`ans = 3`
	`6`
	`9`
		The third column of **A**.
		Explanation: All rows are selected, while only the third column is chosen.

`A(1:2,1:2)`	`ans = 1 2` `4 5`	The sub-matrix consists of the first two rows and columns of **A**. *Explanation*: From the rows we select from the 1st to the 2nd using step 1. Thus, the 1st and 2nd rows are selected. Similarly, only the first two columns are chosen.
`A(1:2:3,1:2:3)`	`ans = 1 3` `7 9`	The sub-matrix contains the first and the third rows and columns of **A**. *Explanation*: From the rows we pick from the 1st to the 3rd using step 2. Thus, the 1st and 3rd rows are selected. Similarly for the columns.

Simple Arithmetic Operations

Working with matrix

`A(1:2:3,1:2)`	`ans = 1 2` `7 8`	The sub-matrix contains the first and the third rows and the first and second columns of **A**. *Explanation*: From the rows we pick from the 1st to the 3rd using step 2. Thus, the 1st and 3rd rows are selected. From the columns we pick from the 1st to the 2nd using step 1. Thus, the 1st and 2nd columns are selected.
`A(1:2:3,:)`	`ans = 1 2 3` `7 8 9`	The sub-matrix contains the first and the third rows of **A**. *Explanation*: From the rows we pick from the 1st to the 3rd using step 2. Thus, the 1st and 3rd rows are selected. Using the colon operator ":" all columns are selected.

`A(:)`	`ans = 1`
	`4`
	`7`
	`2`
	`5`
	`8`
	`3`
	`6`
	`9`
		The statement `A(:)` returns the matrix **A** as a column vector. The elements are sorted by column.

Simple Arithmetic Operations

Addition/ subtraction in matrix

Commands	Results	Comments
A= [1 2 3;4 5 6; 7 8 9] B= [1 1 1; 3 3 3; 5 5 5]	A = 1 2 3 4 5 6 7 8 9 B = 1 1 1 3 3 3 5 5 5	Matrices **A** and **B** of size 3×3.
C=A+B	C = 2 3 4 7 8 9 12 13 14	Matrix addition.
D=A−B	D = 0 1 2 1 2 3 2 3 4	Matrix subtraction.

Simple Arithmetic Operations

Multiplication in matrix

Commands	Results	Comments

`A= [1 2 3; 4 5 6]`
`B= [1 1 1 1;2 2 2 2;3 3 3 3]`
	`A = 1 2 3`
	`4 5 6`
	`B = 1 1 1 1`
	`2 2 2 2`
	`3 3 3 3`
		Matrix **A** is of size 2×3, while matrix **B** is of size 3×4.

`C=A*B`	`C = 14 14 14 14` `32 32 32 32`	Matrix **C** is the product $\mathbf{A} \cdot \mathbf{B}$ and its size is 2×4.
`D=B*A`	`??? Error using ⇒ * Inner matrix dimensions must agree.`	The product $\mathbf{B} \cdot \mathbf{A}$ is not a valid operation as it does not fulfill condition
`A= [1 2 3; 4 5 6; 7 8 9];` `B= [3 2 1; 6 5 4; 9 8 7];`		Matrices **A** and **B** of size 3×3.

`A*B`	`ans = 42 36 30`
	`96 81 66`
	`150 126 102`
		The product $\mathbf{A} \cdot \mathbf{B}$.

`B*A`	`ans = 18 24 30`
	`54 69 84`
	`90 114 138`
		$\mathbf{B} \cdot \mathbf{A}$ is not equal to $\mathbf{A} \cdot \mathbf{B}$.

Simple Arithmetic Operations

Inverse matrix

Commands	Results	Comments

`A= [2 4 6;1 1 1;3 4 1]`	`A = 2 4 6`
	`1 1 1`
	`3 4 1`
		Matrix A of size 3×3.

`B=inv(A)`	`B = -0.3750 2.5000 -0.2500`
	`0.2500 -2.0000 0.5000`
	`0.1250 0.5000 -0.2500`
		The inverse matrix of **A**.

`C=A^-1`	`C = -0.3750 2.5000 -0.2500`
	`0.2500 -2.0000 0.5000`
	`0.1250 0.5000 -0.2500`
		Second way of computing the inverse matrix $\mathbf{A}^{-1}$.

`A*B`	`ans = 1.0000 0.0000 0`
	`0.0000 1.0000 0`
	`0.0000 0.0000 1.0000`
		Indeed the product $\mathbf{A} \cdot \mathbf{A}^{-1}$ is equal to the identity matrix **I**.

`B*A`	`ans = 1.0000 0.0000 0.0000`
	`0.0000 1.0000 0.0000`
	`0.0000 0 1.0000`
		In this case the commutative property stands, that is, $\mathbf{A} \cdot \mathbf{A}^{-1} = \mathbf{A}^{-1} \cdot \mathbf{A} = \mathbf{I}$.

Simple Arithmetic Operations

Transpose of the matrix

Commands	Results	Comments

`A=[1 2 3; 4 5 6;7 8 9];`	`A = 1 2 3`
	`4 5 6`
	`7 8 9`
		Matrix **A** of size 3×3.

`B=A'`	`B = 1 4 7`
	`2 5 8`
	`3 6 9`
		The transposed matrix $\mathbf{A}^T$.

`B'`	`ans = 1 2 3`
	`4 5 6`
	`7 8 9`
		Transposing $\mathbf{A}^T$ results the initial matrix **A**.

`A=[1 2 3; 4 5 6]`	`A = 1 2 3` `4 5 6`	Matrix **A** of size 2×3.

`B=A'`	`B = 1 4`
	`2 5`
	`3 6`
		The transpose of **A** is of size 3×2.

Simple Arithmetic Operations

Special forms of matrix

Commands	Results	Comments
`ones(2,3)`	`ans = 1 1 1` `1 1 1`	Matrix of size 2 × 3 with ones.
`zeros(1,4)`	`ans = 0 0 0 0`	Matrix of size 1 × 4 (or vector of length 4) with zeros.

`rand(3)`	`ans = 0.9501 0.4860 0.4565`
	`0.2311 0.8913 0.0185`
	`0.6068 0.7621 0.8214`
		Matrix of size 3 × 3 with random elements. If there is one input argument to the command, the obtained matrix is square.

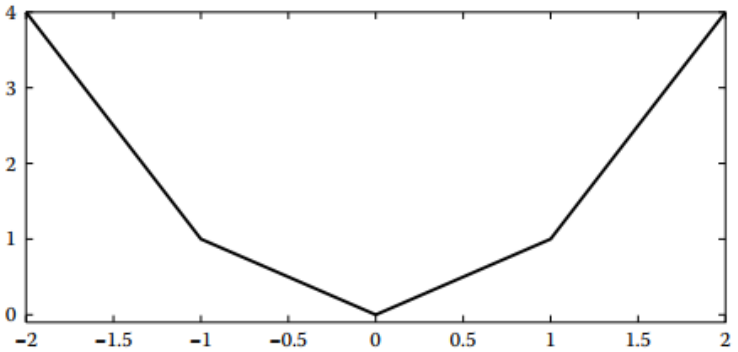
`eye(4,2)`	`ans = 1 0`
	`0 1`
	`0 0`
	`0 0`
		Matrix of size 4 × 2 with ones in the main diagonal and zeros elsewhere.

`eye(3)`	`ans = 1 0 0`
	`0 1 0`
	`0 0 1`
		The identity matrix **I** of size 3 × 3.

Plotting in Matlab

Plotting of functions

Plot the function $y(x) = x^2$, $-2 \leq x \leq 2$.

Commands	Results	Comments
`x=-2:2`	`x = -2 -1 0 1 2`	Vector x is defined from -2 to 2 with step 1.
`length(x)`	`ans = 5.00`	Vector x has 5 elements.
`y=x.^2`	`y = 4 1 0 1 4`	Vector y is created. Notice that $y = [(-2)^2\ (-1)^2\ 0^2\ 1^2\ 2^2]$. Also notice the dot operator which is inserted before the power operator to ensure that this is an element per element operation.
`length(y)`	`ans = 5.00`	Vector y has the same number of elements to x.
`plot(x,y)`		Graph of the function $y(x) = x^2$, $-2 \leq x \leq 2$.

Plotting in Matlab

Plotting of functions

Commands	Results	Comments

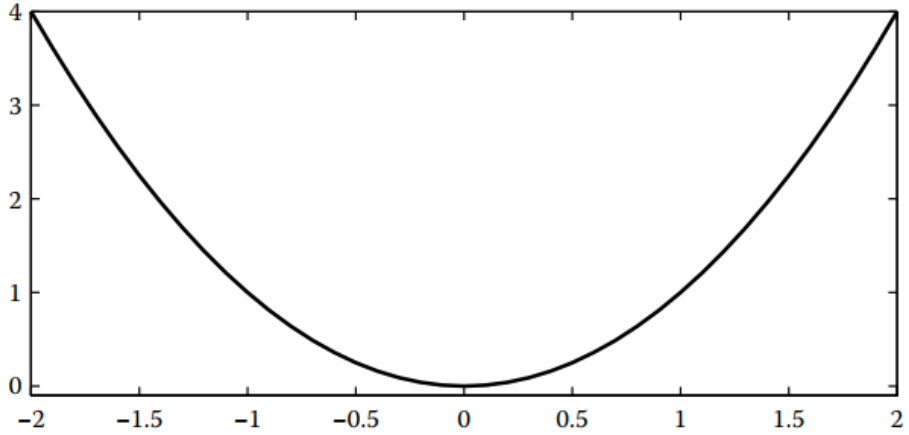
`x=-2:0.1:2`	`x = -2.00 -1.90 -1.80 -1.70 -1.60`
	`-1.50 -1.40 -1.30 -1.20 -1.10`
	`-1.00 -0.90 -0.80 -0.70 -0.60`
	`-0.50 -0.40 -0.30 -0.20 -0.10`
	`0 0.10 0.20 0.30 0.40 0.50 0.60`
	`0.70 0.80 0.90 1.00 1.10 1.20`
	`1.30 1.40 1.50 1.60 1.70 1.80`
	`1.90 2.00`
		The vector x is defined from -2 to 2 with step 0.1.

`plot(x,y)`	`??? Error using ⇒ plot Vectors must be the same lengths.`	Trying to execute the command `plot` we get an error warning as `length(x) = 41` and `length(y) = 5`.
`y=x.^2;`		Vector y has to be redefined according to the new vector x.
`length(y)`	`ans = 41.00`	Now the two vectors have the same number of elements and the function $y(x) = x^2$, $-2 \leq x \leq 2$ can be plotted.

Plotting in Matlab

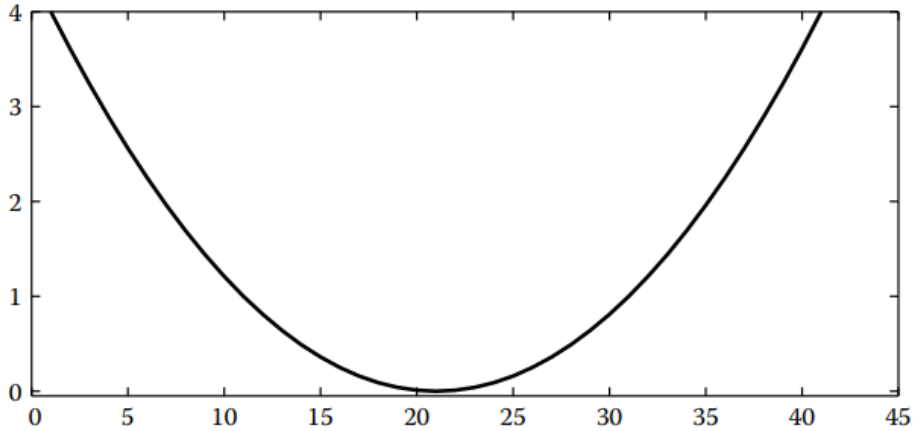
Plotting of functions

`plot(x,y)`



This time the graph is satisfactory.

`plot(y)`



It is possible to execute the command `plot` with only one input argument. The result appears to be correct. However, the x-axis is not the interval [−2 2] but [1 41]. More specifically, each element of y (recall that y consists of 41 elements) is plotted versus its index in the vector y and not versus the corresponding element of x.

Plotting in Matlab

Linspace command

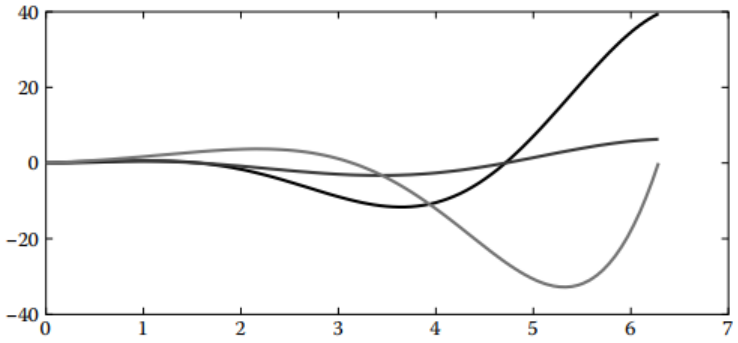
Commands	Results	Comments
`x=linspace(0,1,5)`	`ans = 0 0.25 0.50 0.75 1.00`	Vector x from 0 to 1 with 5 elements.
`x=0:1/4:1`	`ans = 0 0.25 0.50 0.75 1.00`	Equivalent vector creation.
`logspace(0,1,5)`	`ans = 1.00 1.78 3.16 5.62 10.00`	The command `logspace` creates 5 logarithmically spaced points on the interval $[10^0\ 10^1]$.

Plotting in Matlab

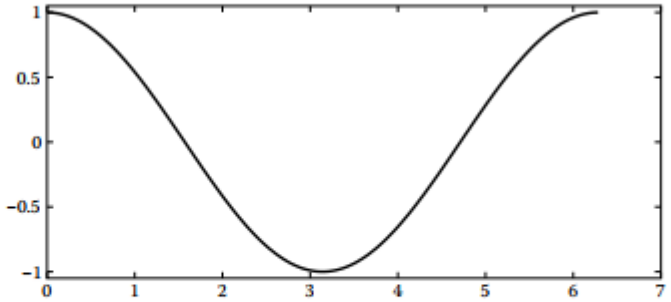
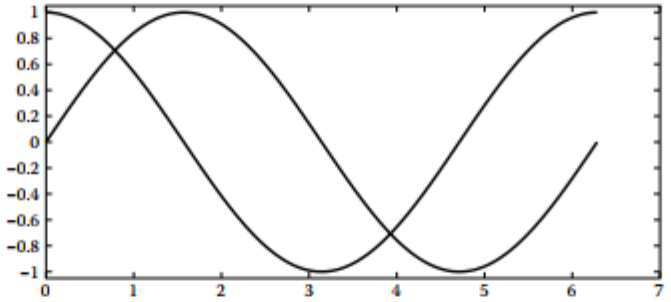
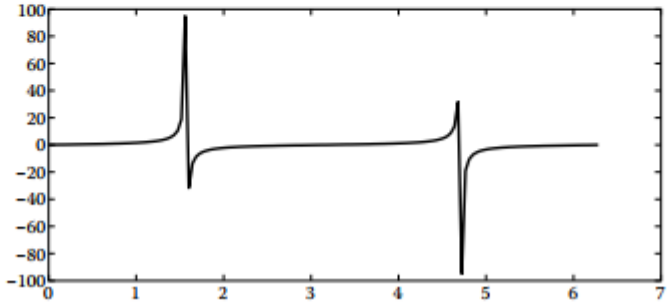
Plotting several functions in one plot

Commands	Results	Comments
`x=linspace(0,2*pi,100);`		Definition of vector x.

`y=(x.^2).*cos(x);`
`g=x.*cos(x);`
`f=(2.^x).*sin(x);`
		Definition of vectors y, g, and f.

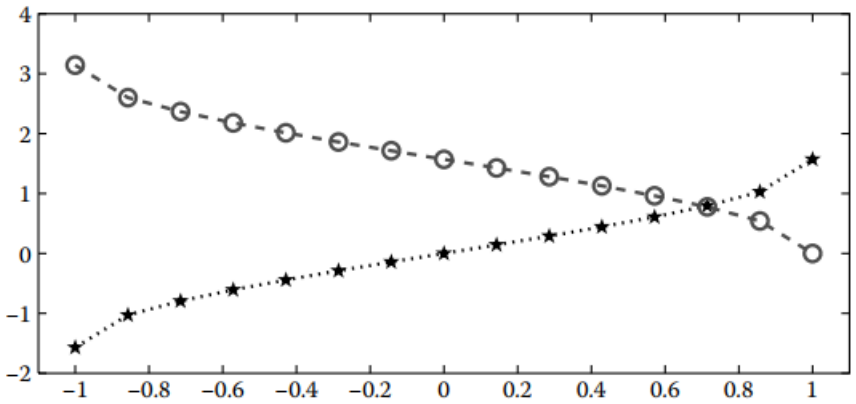
`plot(x,y,x,g,x,f)`		Using this syntax of the command `plot`, it is possible to plot many functions in the same figure. Each function is plotted in different color.

Plotting several functions in one plot

Commands	Results	Comments
`x=linspace(0,2*pi,150);` `plot(x,cos(x));`		Definition of vector x and direct plot of cos (x).
`hold on`		The old graph is preserved.
`plot(x,sin(x));`		The new function is plotted together with the old one.
`hold off`		Switch to the non-hold mode.
`plot(x,tan(x))`		The old graph is discarded and the new function is plotted alone.

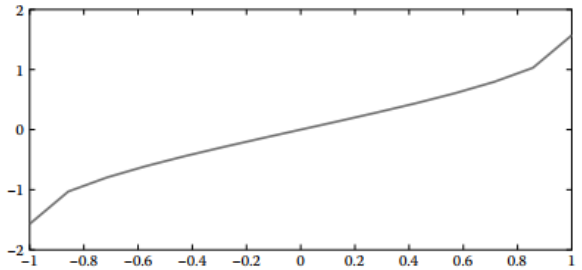
Plotting functions with different symbols or line patterns

Symbol	Color	Symbol	Point Type	Symbol	Line Type
b	Blue	.	Point	-	Solid
g	Green	o	Circle	:	Dotted
r	Red	x	x-mark	-.	Dashdot
c	Cyan	+	Plus	–	Dashed
m	Magenta	*	Star		
y	Yellow	s	Square		
k	Black	d	Diamond		
w	White	<, >	Triangle		
		p	Pentagram		
		h	Hexagram		

Commands	Results	Comments
`x=linspace(-1,1,15);`		Definition of vector x.
`y1=asin(x);` `y2=acos(x);`		Definition of vectors $y1$ and $y2$.
`plot(x,y1,'k:p',x,y2,'r-o')`		The first function is plotted by black pentagons with dotted line, while the second function is plotted by red circles with dashed line.

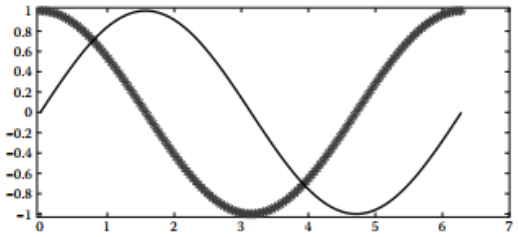
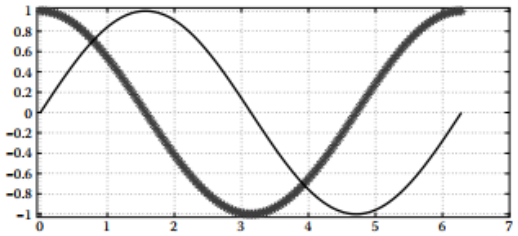
Plotting in Matlab

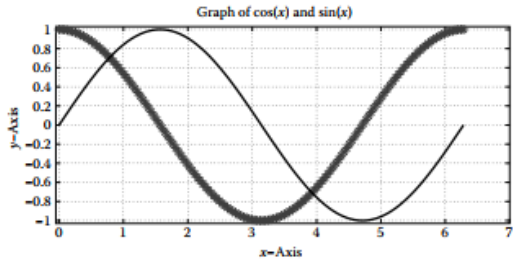
Plotting functions with different symbols or line patterns

Commands	Results	Comments
`plot(x,y1,'Color',[1 0 1], 'Linewidth',4)`		The vector that appears after the text ''Color'' specifies the RGB elements that take values on [0 1]. Number 4 determines the line width.

Plotting in Matlab

Formatting of the figure

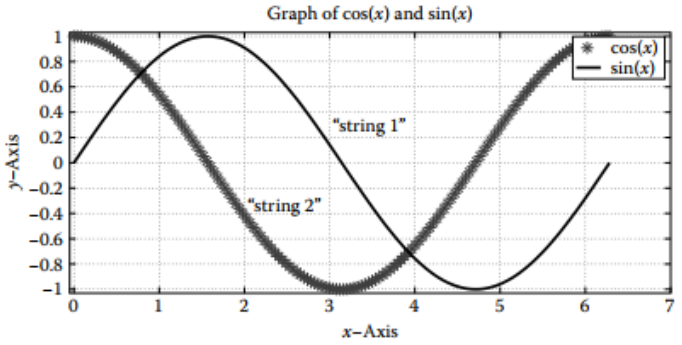
Commands	Results	Comments
`x=linspace(0,2*pi,150);` `plot(x,cos(x),'r*',x,sin(x),'k')`		Definition of the vector x and plot of $\cos(x)$ and $\sin(x)$.
`grid`		Insertion of grid lines.

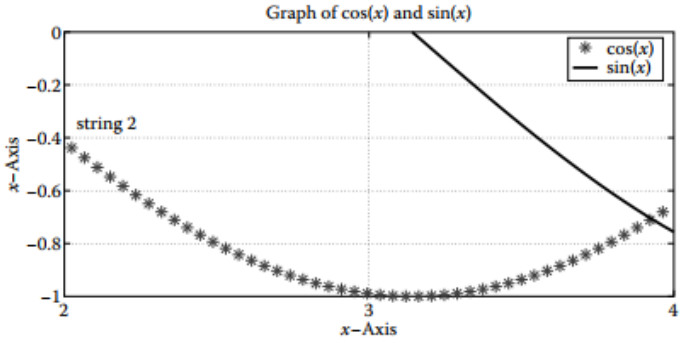
`xlabel('x-axis')`
`ylabel('y-axis')`
`title('Graph of cos(x) and sin(x)')`
		Labels in the axes and graph title.

Plotting in Matlab

Formatting of the figure

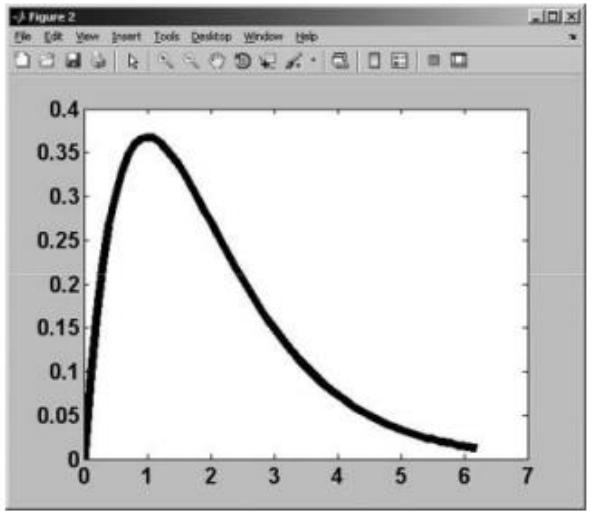
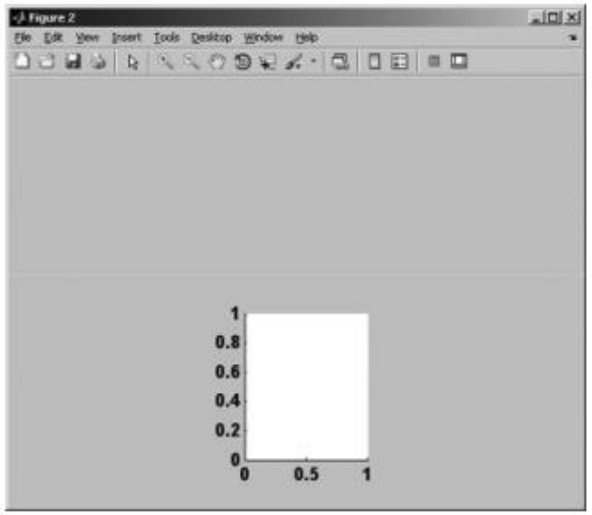
Commands	Results	Comments

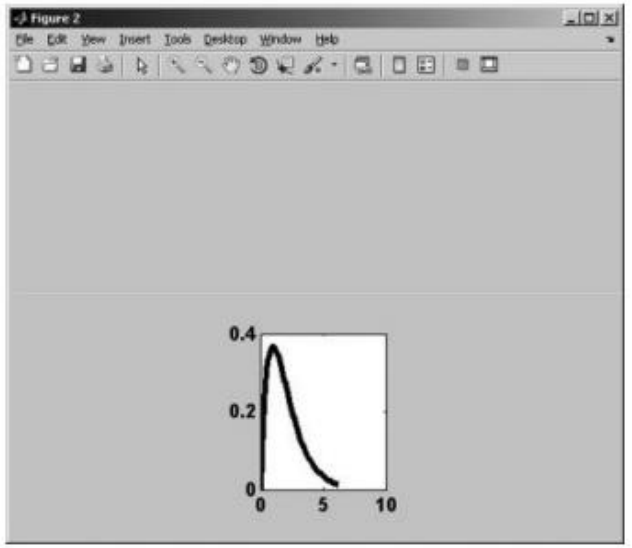
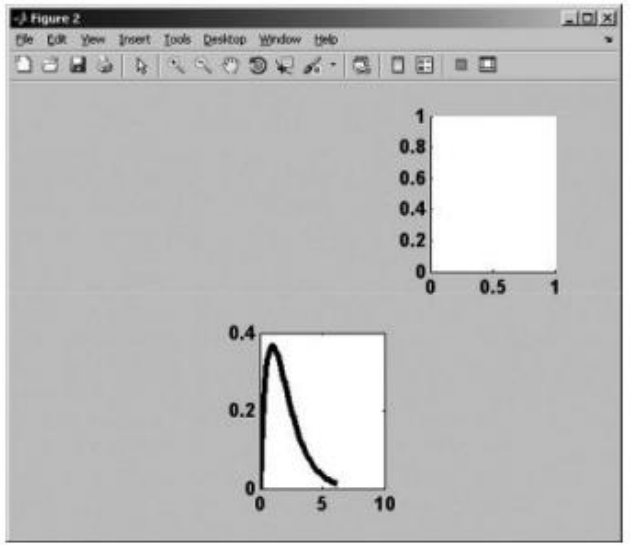
`text(3,0.3,'string1')`
`gtext('string2')`
`legend('cos(x)','sin(x)')`
		The text "string1" is placed at the point with coordinates [3 0.3], while the text "string2" is placed with the mouse. The `legend` command clarifies which function corresponds to which curve. The legend is by default placed at the upper left part of the graph.

`axis([2,4,-1,0])`		Change of limits. The x-axis is at [2 4] and the y-axis is at [−1 0].

Plotting in different figure

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

Commands	Results	Comments
`x=0:.1:2*pi;` `plot(x,x.*exp(-x));`		The window *Figure 2* is the active one, i.e., the next graph is plotted in it.
`subplot(2,3,5)`		The figure is split in $2 \times 3 = 6$ subfigures. The active subfigure is the fifth.

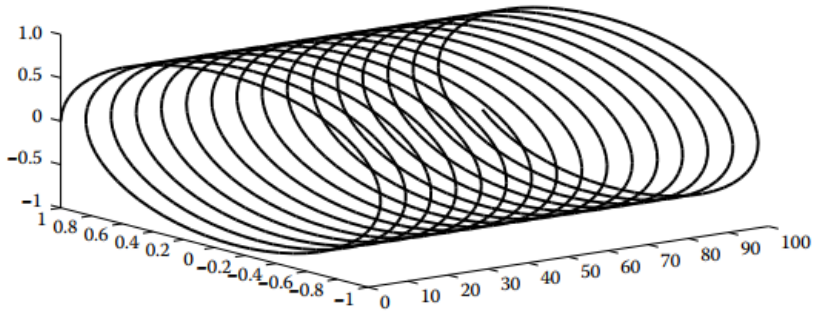
Commands	Results	Comments
`plot(x,x.*exp(-x))`		Plotting in the active subfigure.
`subplot(2,3,3)`		Activation of the third subfigure.

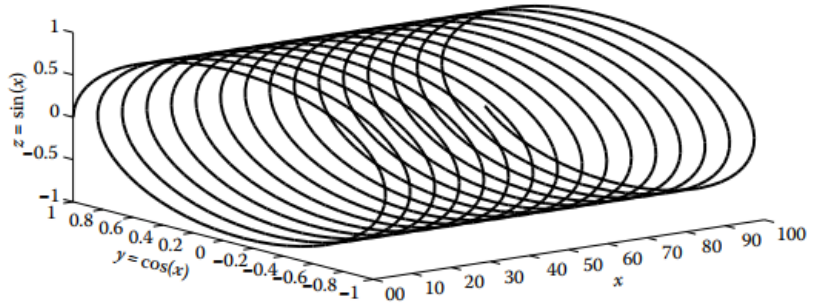
Plotting in Matlab

Plotting in 3 dimensions

Commands	Results	Comments

`x=0:.1:100;`
`y=cos(x);`
`z=sin(x);`
		Definition of the function $f(x) = (x, \cos(x), \sin(x))$, $0 \leq x \leq 100$.

`plot3(x,y,z)`		Graph of $f(x,y,z)$ in the 3-D space.

`xlabel('x')`
`ylabel('y=cos(x)')`
`zlabel('z=sin(x)')`
		Labels in the three axes.

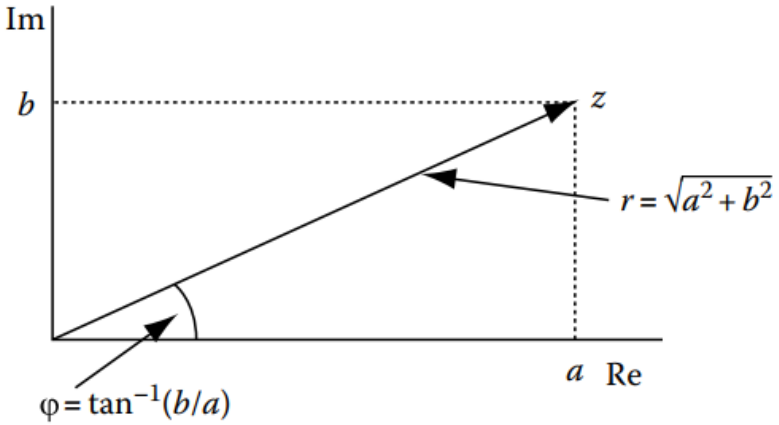
Useful Commands

Real and complex

Commands	Results	Comments

`a=4;`
`b=3;`
`z=a+b*i`
	`z = 4.0000 + 3.0000i`	Complex number $4+3i$.

`real(z)`	`ans = 4`	Real part of z.
`imag(z)`	`ans = 3`	Imaginary part of z.
`abs(z)`	`ans = 5`	Magnitude of z.
`sqrt(a^2+b^2)`	`ans = 5`	Indeed, the magnitude is given by $\|z\| = \sqrt{a^2+b^2}$.
`angle(z)`	`ans = 0.6435`	The angle of z in radians.
`phase(z)`	`ans = 0.6435`	Alternative computation of the angle.
`atan(b/a)`	`ans = 0.6435`	Indeed, the angle of z is given by $\phi = \tan^{-1}(b/a)$.
`angle(z)*180/pi`	`ans = 36.8699`	Conversion from radians to degrees.
`conj(z)`	`ans = 4.0000 - 3.0000i`	Complex conjugate.
`z'`	`ans = 4.0000 - 3.0000i`	Alternative computation of the complex conjugate.
`z*conj(z)`	`ans = 25`	An important property of complex numbers is revealed here. More specifically $zz^* = \|z\|^2$, where $\|z\|$ is the magnitude of z.



Useful Commands

Forms of complex numbers

Commands	Results	Comments
`z=4+3i`	`z = 4.0000 + 3.0000i`	Complex number $4+3i$.
`A=abs(z)`	`A = 5`	Magnitude of z.
`fi=angle(z)`	`fi = 0.6435`	Angle of z.
`z1=A*exp(i*fi)`	`z1 = 4.0000 + 3.0000i`	Exponential equivalent form of z.
`A*cos(fi)`	`ans = 4`	Indeed $a = A\cos(\varphi)$.
`A*sin(fi)`	`ans = 3`	And also $b = A\sin(\varphi)$.
`A*(cos(fi)+i*sin(fi))`	`ans = 4.0000 + 3.0000i`	Trigonometric equivalent form of z.
`z3=exp(i)`	`z3 = 0.5403 + 0.8415i`	The term $z3 = e^i$ is $0.5403 + 0.8415$i.
`abs(z3)`	`ans = 1`	The magnitude of $z3$ is $A = 1$.
`angle(z3)`	`ans = 1`	The angle of $z3$ is $\varphi = 1$ rad.

Useful Commands

Operations with complex numbers

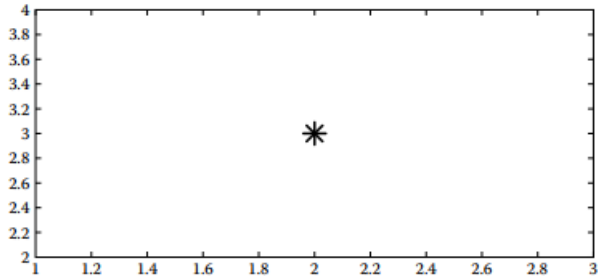
Commands	Results	Comments
`z1=1+2i` `z2=2+1i`	`z1 = 1.0000+2.0000i` `z2 = 2.0000+1.0000i`	Definition of the complex numbers z1 and z2.
`z1+z2`	`ans = 3.0000+3.0000i`	Sum of complex numbers.
`z1-z2`	`ans = -1.0000+1.0000i`	Subtraction of complex numbers.
`z1*z2`	`ans = 0+5.0000i`	Product of complex numbers.
`z1/z2`	`ans = 0.8000+0.6000i`	Division of complex numbers.
`z1^2`	`ans = -3.0000+4.0000i`	Power of a complex number.
`a=[z1 z2]`	`a = 1.00+2.00i  2.00+1.00i`	Row vector of complex numbers

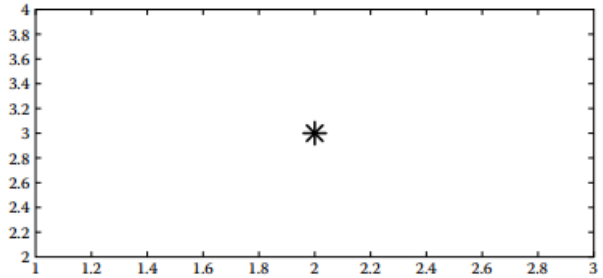
`b=[z1;z2;z1]`	`b = 1.0000+2.0000i`
	`2.0000+1.0000i`
	`1.0000+2.0000i`
		Column vector of complex numbers.

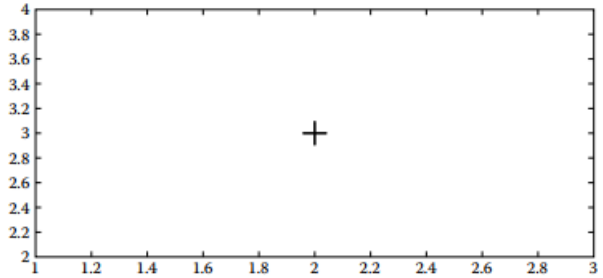
`B=[z1 z2; z2 z1]`	`B = 1.00+2.00i  2.00+1.00i` `2.00+1.00i  1.00+2.00i`	Matrix of complex numbers.

Useful Commands

Graphs of complex numbers

Commands	Results	Comments
`z1=2+3i`	`z1 = 2.0000+3.0000i`	Definition of z1.
`plot(z1,'*')`		Graph of z1 in the complex plane with a star. The star is placed at the coordinates [2, 3].

`a=real(z1);`
`b=imag(z1);`
`plot(a,b,'*')`
		This analytical way is given in order to understand that plotting in the complex plane is a graph of the real versus the imaginary part of a complex number.

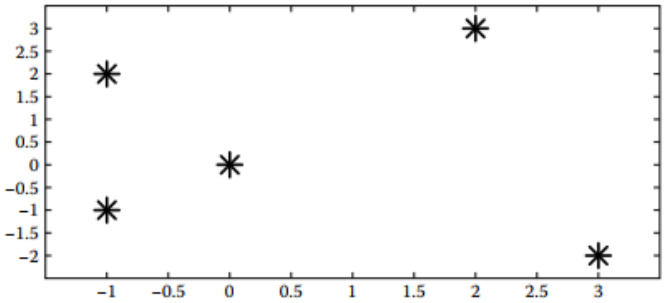
`plot(real(z1),imag(z1),'+')`		Plot of z1 in the complex plane with a cross.

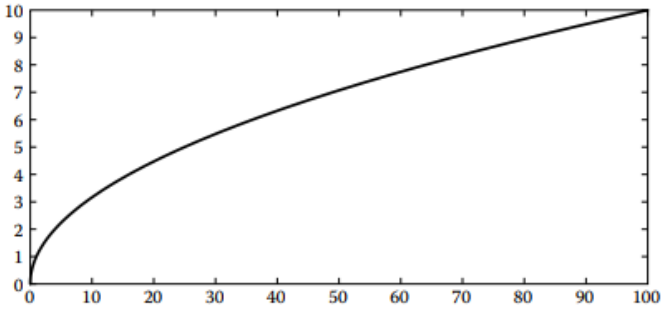
Useful Commands

Graphs of complex numbers

Commands	Results	Comments

`z1=2+3i;`
`z2=3-2i;`
`z3=-1+2i;`
`z4=-1-i;`
`z5=0+0i;`
`z=[z1 z2 z3 z4 z5]`
	`z = 2.00+3.00i  3.00-2.00i`
	`-1.00+2.00i  -1.00-1.00i`
	`0`
		A vector of complex numbers.

`plot(z,'*')` `axis([-1.5 3.5 -2.5 3.5])`		Graph of the vector elements in the complex plane.

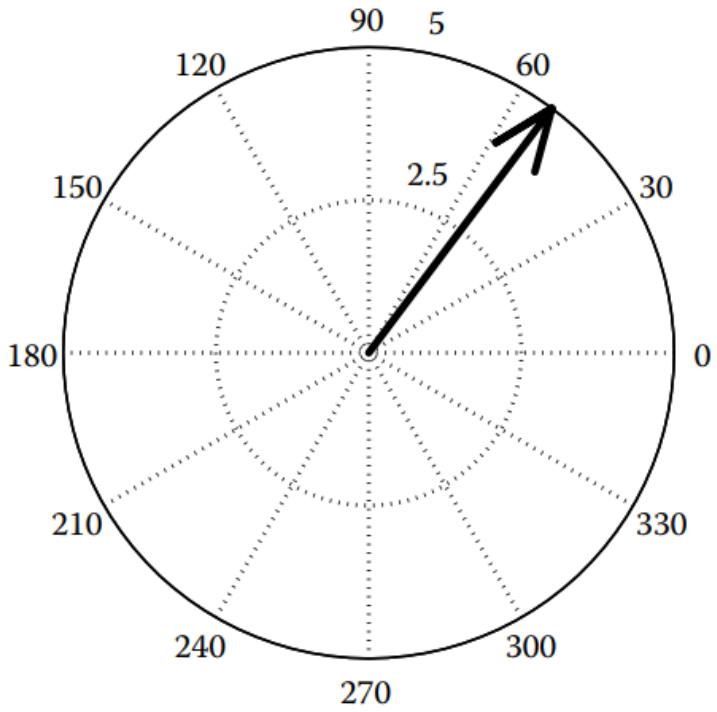
`x=0:0.1:10`
`z=x.^2+x*i`
`plot(z)`
		Plotting with line a vector of complex elements.

Useful Commands

Graphs of complex numbers

Commands	Results	Comments

`z=3+4i;`
`mag= abs(z)`
`ang=angle(z)*180/pi`
	`mag = 5`
	`ang = 53.1301`
		The magnitude of z is 5 and the angle is approximately 53°.

`compass(z)`		Indeed, the magnitude of the arrow is 5 and the angle is 53.1301°.

Useful Commands

Logical operators

Relational Operator	Command	Explanation	Examples	
==	eq	Equal	20==20 ans = 1	eq(20,20) ans = 1
>	gt	Greater than	3>5 ans = 0	gt(5,3) ans = 1
<	lt	Less than	3<5 ans = 1	lt(5,3) ans = 0
~=	ne	Not equal	2~=2 ans = 0	ne(2,3) ans = 1
>=	ge	Greater than or equal	5>=5 ans = 1	ge(4,5) ans = 0
<=	le	Less than or equal	5<=4 ans = 0	le(5,5) ans = 1

Useful Commands

Logical operators

Operator	Explanation	Example
`&`	Logical operator AND.	`1&0` `ans = 0`
`|`	Logical operator OR.	`1|0` `ans = 1`
`~`	Logical operator NOT.	`~0` `ans = 1`
`xor()`	Logical operator XOR.	`xor(1,1)` `ans = 0`

`all (condition)`	Returns 1 if the condition is fulfilled for all elements of the vector.	`a= [1 2];`
		`b= [3 2];`
		`all(a<b)`
		`ans = 0`

`any(condition)`	Returns 1 if the condition is fulfilled for any element of the vector.	`any(a<b)` `ans = 1`

Useful Commands

Control flow (if else)

```
if condition1
   Statements
elseif condition2
   Statements
else
   Statements
end
```

M-Files and Their Execution from the Command Prompt	Results	Comments

`a=input('mark?')`
`if (a>0 &a<5)`
`  disp('fail')`
`elseif (a>=5 & a<=10)`
`  disp('pass')`
`else`
`  disp('choose again')`
`end`
	`mark ? 3`
	`fail`
		The statements `if-elseif-else`.
		The user input is number 3. Notice the use of the logical operator & (i.e., AND).

Useful Commands

Control flow (switch case)

```
switch (variable)
case {value1}
  Statements
case {value2,value3,...}
  Statements
otherwise
  Statements
end
```

```
a = input ('mark?')
switch (a)
   case{1,2,3,4}
       disp('fail')
   case{5,6,7,8,9,10}
       disp('pass')
   otherwise
       disp('choose again')
 end
```

```
mark ? 8
pass
```

The statement `switch`. Program *swi.m* is equivalent to the two previous implementations.

Useful Commands

Control flow (if else with logic gate)

```
if condition1
  Statements
elseif condition2
  Statements
else
  Statements
end
```

```
a = input ('mark?')
if (a>0 &a<5)
   disp('fail')
end
if (a>=5 & a<=10)
    disp('pass')
end
if (a<=0 |a>10)
    disp('choose again')
end
```

```
mark ?16
choose again
```

The requested program is implemented using the simple syntax of the `if` statement. Notice the use of the logical operator | (i.e., OR).

Useful Commands

Control flow (for loop)

The command `for` is used to repeat statements a specific number of times. The syntax is

```
for counter = vector
Statements
end
```

```
for i=1:5
  b(i)=i^2;
end
display(b)
```

```
b = 1    4    9    16    25
```

Loop implemented with the command `for`. The comments at the first 2 lines illustrate an alternative way to define the loop-counter *i*. In this program, a for-loop is used to create a vector whose elements are the squares of the numbers 1 to 5

Useful Commands

Control flow (while loop)

The command `while` is used to repeat statements an indefinite number of times. The syntax is

```
while (condition)
Statements
end
```

```
a=1;
while(a>=0)
   disp('positive to loop')
   disp('negative to exit')
   a=input('enter number');
 end
```

```
positive to loop, negative to exit
enter number 89
positive to loop, negative to exit
enter number 2
positive to loop, negative to exit
enter number -5
```

Loop implemented with command `while`. Notice that variable *a* must be initialized before being used in the loop. In this program, the loop continues as long as the user input is not a negative number.

Useful Commands

Control flow (nested loop)

```
a=1;
while(1)
    a=input('Mark ?')
    if (a>0 & a<5)
        disp('fail')
        break;
     elseif (a>=5 & a<=10)
        disp('pass')
        break;
     else
        disp('choose again');
     end
end
```

```
Mark ? 11
choose again
Mark ? -6
choose again
Mark ? 7
pass
```

By typing `while(1)` an infinite loop is created. In order to terminate the infinite loop the command `break` is used. In this program, if the user input is not valid (i.e., between 1 and 10) the user is requested to enter again his mark.

Useful Commands

Differentiation and integration

Commands	Results	Comments
`syms x y`		Declaration of the symbolic variables x and y.
`f=exp(-x)*cos(y)`	`f = exp(-x)*cos(y)`	The function $f(x, y)$ is a symbolic expression.
`diff(f,x)`	`ans = -exp(-x)*cos(y)`	Partial derivative $\partial f/\partial x$.
`diff(f,y)`	`ans = -exp(-x)*sin(y)`	Partial derivative $\partial f/\partial y$.
`diff(f,x,2)`	`ans = exp(-x)*cos(y)`	Second derivative of f with respect to x.

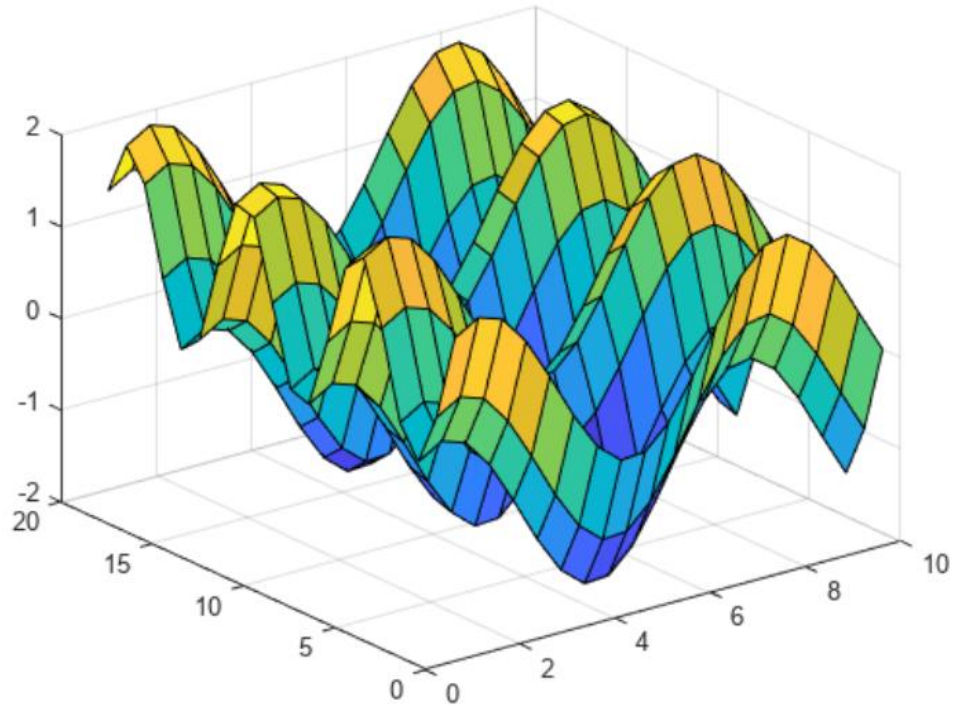
Commands	Results	Comments
`syms x t`		Declaration of the symbolic variables x and t.
`f=x^2` `g=x^2+exp(-t)`	`f=x^2` `g=x^2+exp(-t)`	The functions $f(x) = x^2$ and $g(x,t) = x^2 + e^{-t}$ are defined as symbolic expressions.
`int(f,x)`	`ans = 1/3*x^3`	Computation of $\int x^2 dx$.
`pretty(ans)`	$1/3\ x^3$	The command `pretty` prints symbolic output in a format that resembles typeset mathematics.

Useful Commands

Plotting of 3D Graphs

Create three matrices of the same size. Then plot them as a surface. The surface plot uses Z for both height and color.

```
[X,Y] = meshgrid(1:0.5:10,1:20);
Z = sin(X) + cos(Y);
surf(X,Y,Z)
```



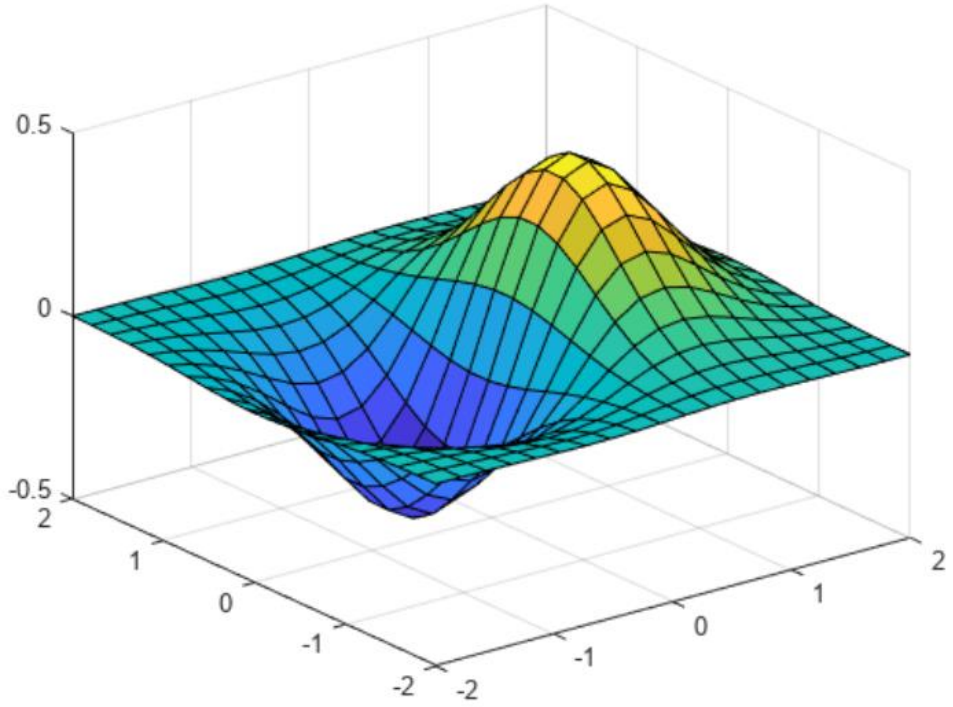
Useful Commands

Plotting of 3D Graphs

Three-dimensional plots typically display a surface defined by a function in two variables, $z = f(x, y)$.

For instance, calculate $z = xe^{-x^2-y^2}$ given row and column vectors x and y with 20 points each in the range [-2,2].

```
x = linspace(-2,2,20);
y = x';
z = x .* exp(-x.^2 - y.^2);
surf(x,y,z)
```



Useful Commands

Basic Fitting Tool

```
%Plotting of data
x=[0 5 2 4 3];
y=[1 2 3 4 5];
plot(x,y);

%From Figure Window
%> Tools > Basic Fitting
```

